THIAGARAJAR COLLEGE OF ENGINEERING, MADURAI

(An Autonomous Institution, Affiliated to Anna University, Chennai)

DEPARTMENT OF INFORMATION TECHNOLOGY

A PROJECT REPORT

Submitted in partial fulfilment of the requirements of the course

LARGE LANGUAGE MODELS

CIVICASSIST

Local Government Scheme Assistant using Private RAG & SLM/LLM Inference

Submitted by

Anish M – 2303917720521015

Moulesh G S – 2303917720521087

Prasanna T – 2303917720521107

Yogesh T – 2303917720521177

BACHELOR OF TECHNOLOGY

IN

INFORMATION TECHNOLOGY

September 2026

Abstract

Access to government welfare schemes remains a persistent challenge for citizens across India, primarily due to the fragmentation of scheme information across hundreds of official portals, the use of complex bureaucratic language in eligibility documentation, and the multi-variate nature of eligibility determination that spans income, social category, age, geography, and occupation. Compounding this problem, general-purpose conversational AI systems that could otherwise simplify this discovery process introduce two further risks: they are prone to hallucinating non-existent schemes or incorrect eligibility criteria, and they typically require citizen data to be transmitted to third-party cloud infrastructure, raising serious privacy concerns for sensitive demographic and financial information.

This project presents CIVICASSIST, a local-first, privacy-preserving welfare scheme discovery and eligibility-matching platform designed to address these gaps without reliance on any paid cloud API. The system is built around a Retrieval-Augmented Generation (RAG) pipeline that combines dense semantic search using FAISS (Facebook AI Similarity Search) with embeddings generated by the sentence-transformers/all-MiniLM-L6-v2 model, and a structured, rule-based eligibility scoring engine that evaluates citizen profiles against scheme metadata using weighted factors such as need category, social category, geographic alignment, income, and age. Natural-language response generation is handled entirely on-device using locally hosted Small and Large Language Models served through Ollama — specifically Phi-3 Mini, Gemma 2B, and Llama 3.2 — allowing the system to operate on consumer-grade, CPU-only hardware with as little as 4 GB of RAM.

A key architectural contribution of CIVICASSIST is its hybrid execution strategy, which allows the system to operate in three distinct modes: a Small Language Model (SLM) mode optimized for low latency and concise responses, a Large Language Model (LLM) mode optimized for richer conversational synthesis and multi-step reasoning, and a Retrieval-Only fallback mode that returns deterministic, zero-hallucination scheme cards directly from the verified knowledge base. This design ensures that only eligibility-verified scheme matches, filtered through a two-stage retrieval-and-scoring pipeline, are ever passed to the language model for explanation, thereby substantially reducing the risk of hallucinated or fabricated recommendations — a critical requirement for any system operating in the civic and public-welfare domain.

The system was evaluated using representative citizen profiles covering agricultural, senior-citizen, and student-minority personas, and its performance was benchmarked across the three execution modes on the basis of response latency, factual groundedness, hardware resource requirements, and comparative cost against commercial cloud LLM APIs. The results demonstrate that local SLM/LLM inference combined with structured retrieval is a technically and economically viable alternative to cloud-dependent civic-tech deployments, particularly for resource-constrained environments such as rural digital service centers and Common Service Centers (CSCs), while preserving complete citizen data privacy by ensuring that no personal information ever leaves the local device.

The remainder of this report details the problem analysis and existing solution gaps, a comparative study of Small versus Large Language Models, the system architecture and workflow, the implementation methodology, experimental results, and a discussion of the advantages, limitations, and future scope of the CIVICASSIST platform.

Keywords: Retrieval-Augmented Generation (RAG), Small Language Models (SLM), Large Language Models (LLM), FAISS, Sentence Transformers, Ollama, Local Inference, Government Welfare Schemes, Eligibility Matching, Privacy-Preserving AI.

1. Introduction

1.1 Background

Government welfare schemes form a critical component of social security infrastructure, providing financial assistance, housing support, healthcare coverage, pensions, and educational aid to eligible citizens. In India, welfare delivery spans a large number of Central and State-level schemes, each governed by its own eligibility rules, application procedures, and official information portals. While digital initiatives such as centralized scheme-discovery portals (e.g., MyScheme) have been introduced to consolidate this information, citizens — particularly those in rural or semi-urban areas — continue to face significant difficulty in identifying which schemes they qualify for and how to apply.

The CIVICASSIST project was undertaken to explore whether locally deployed Small and Large Language Models, combined with a structured retrieval and eligibility-matching pipeline, could make this scheme-discovery process more accessible, accurate, and privacy-preserving, without requiring citizens or civic-technology deployers to depend on paid cloud AI infrastructure.

1.2 Problem Statement

Despite the existence of numerous government welfare schemes, a substantial number of eligible citizens fail to access the benefits available to them. This gap arises from four interlinked problems identified in the project analysis:

- Information Fragmentation: Central and State scheme information is scattered across hundreds of official websites, with no unified, conversational access point.
- Complex Eligibility Rule Matching: Determining eligibility requires the simultaneous evaluation of multiple variables — income, social category, age, land holding, and state of residence — which is difficult for citizens to self-assess.
- Hallucination Risk in Public AI: General-purpose conversational AI systems frequently hallucinate non-existent schemes or state incorrect deadlines and criteria, which is particularly harmful in a civic-welfare context where inaccurate information can cause citizens to miss genuine entitlements.

- Digital Divide and Infrastructure Limits: Rural service centers typically lack the high-bandwidth internet connectivity and budget required to sustain recurring cloud API costs, limiting the deployability of cloud-dependent AI assistants in these settings.

1.3 Motivation

The motivation for CIVICASSIST stems directly from the limitations observed in existing approaches to scheme discovery. Government portals such as MyScheme provide static, filter-based search but lack conversational understanding and the ability to synthesize eligibility across multiple criteria. Commercial cloud-based LLMs such as ChatGPT and Claude offer strong conversational capability but require high-bandwidth internet access, paid API subscriptions, and — critically — involve transmitting sensitive citizen demographic and financial data to third-party servers, which raises data-privacy concerns for a domain that inherently deals with personal and economic information. Generic Retrieval-Augmented Generation pipelines, while capable of grounding responses in a knowledge base, generally lack a dedicated structured eligibility-calculation engine that can quantitatively score a citizen's likelihood of qualifying for a given scheme. CIVICASSIST is motivated by the opportunity to bridge this three-way gap by combining FAISS-based vector search, a rule-based eligibility engine, and local SLM/LLM inference into a single, zero-cloud-dependency platform.

1.4 Objectives

- To design and implement a local-first Retrieval-Augmented Generation pipeline for government welfare scheme discovery that does not depend on any cloud-based API.
- To build a structured, multi-factor rule-based eligibility scoring engine capable of ranking candidate schemes by a citizen's profile likelihood match.
- To integrate locally hosted Small Language Models (SLM) and Large Language Models (LLM) via Ollama, enabling a hybrid execution strategy across SLM, LLM, and Retrieval-Only modes.
- To ensure complete citizen data privacy by guaranteeing that no personal or demographic data leaves the local device during processing.

- To evaluate and compare the performance, factual groundedness, hardware footprint, and cost implications of the different local model configurations against commercial cloud LLM alternatives.

1.5 Scope

The scope of the CIVICASSIST prototype, as demonstrated in the project, covers a curated knowledge base of six representative Central Government welfare schemes — PM-KISAN, Pradhan Mantri Awas Yojana (PMAY), Ayushman Bharat PM-JAY, the National Social Assistance Programme (NSAP), Minority and Merit Scholarships, and Women & Child Development welfare schemes. The system is designed around a Streamlit-based citizen profile interface, a FAISS vector store built using SentenceTransformer embeddings, a rule-based eligibility engine, and local inference through Ollama-hosted SLM/LLM models (Phi-3, Gemma, and Llama 3.2). The current scope is limited to a single-language (English) interface and a fixed scheme corpus; live scraping of scheme portals, multi-lingual support, and mobile deployment are identified as future enhancements (see Chapter 9: Future Work) rather than part of the current implementation.

1.6 Success Criteria

Based on the evaluation metrics defined in the project, the success of CIVICASSIST is measured against the following criteria:

- Eligibility match accuracy: Candidate schemes must achieve a computed match score of at least 55% to be classified as “Recommended” to the citizen.
- Factual groundedness: Model responses should maintain a high groundedness/accuracy score (as benchmarked in Chapter 6), minimizing hallucinated content.
- Resource efficiency: The system should remain operable on CPU-only consumer hardware with as little as 4 GB of RAM in Retrieval-Only mode.
- Zero recurring cost: The platform should eliminate per-token or per-query API charges associated with commercial cloud LLM usage.
- Data privacy: No citizen input data should be transmitted outside the local device at any stage of processing.

2. Literature Review

This chapter reviews the existing categories of solutions in the government-scheme discovery and civic-technology space, as identified through the comparative gap analysis presented in the project.

2.1 Existing Solution Categories

Government Scheme Portals

Centralized government portals, such as MyScheme, aggregate scheme information from multiple departments into a single searchable interface. However, as identified in the project's comparative analysis, these portals are limited to static, filter-based search mechanisms. They lack conversational understanding of a citizen's natural-language description of their needs and do not synthesize eligibility across multiple criteria simultaneously; a citizen must manually cross-reference filters such as category, state, and income bracket to identify relevant schemes.

Commercial Cloud-Based Conversational AI

General-purpose commercial Large Language Models, such as ChatGPT and Claude, are capable of conversational, natural-language interaction and can, in principle, discuss welfare scheme eligibility. However, the project's analysis identifies three limitations specific to this category: a requirement for high-bandwidth internet connectivity, dependency on paid API access, and the exposure of user privacy through the transmission of citizen data to third-party cloud infrastructure. Additionally, as noted in the problem analysis, generic LLMs without domain grounding are prone to hallucinating fake schemes or invalid deadlines, which is a critical failure mode in a civic-welfare context.

Generic Retrieval-Augmented Generation Pipelines

Generic RAG pipelines, which combine a vector-based retrieval mechanism with a language model for answer synthesis, address some of the grounding limitations of standalone LLMs by retrieving relevant context before generation. However, the project identifies that such generic pipelines typically lack a specialized eligibility-calculation engine operating alongside semantic vector search — that is, they may retrieve semantically relevant documents but do not perform structured, multi-factor eligibility scoring (income thresholds, categorical matching, age-based rules, etc.) in conjunction with retrieval.

2.2 Research Gap

Based on the comparative gap analysis in the source project material, the research gap addressed by CIVICASSIST can be summarized as the absence of a single system that simultaneously provides: (i) conversational, natural-language understanding of citizen needs; (ii) grounded, hallucination-resistant responses through retrieval-augmented generation; (iii) structured, multi-factor eligibility computation; and (iv) complete data privacy through local, offline-capable inference at zero recurring cost. CIVICASSIST is positioned to bridge this gap by combining FAISS vector search, a structured rule-based eligibility engine, and local SLM/LLM inference within a single pipeline.

2.3 Additional Literature

A comprehensive academic literature review — covering peer-reviewed publications on Retrieval-Augmented Generation architectures, comparative studies of Small versus Large Language Model deployment, prior civic-technology and e-governance research, and formal evaluations of hallucination mitigation techniques .

3. Comparative Analysis of SLM vs LLM

A central design decision in CIVICASSIST is the choice between Small Language Models (SLM) and Large Language Models (LLM) for local, on-device response generation. This chapter presents a comparative analysis of the two categories, along with the Retrieval-Only fallback mode, based on the models evaluated in the project: Phi-3 Mini and Gemma 2B (SLM category), and Llama 3.2 and Qwen 2.5 (LLM category).

3.1 Small Language Models (SLM)

Small Language Models, represented in this project by Phi-3 Mini (3.8B parameters) and Gemma 2B, are characterized by a substantially smaller parameter count relative to general-purpose LLMs. In the CIVICASSIST evaluation, SLMs demonstrated a RAM footprint of under 4–8 GB, high inference speed with response latency in the range of approximately 1.4 to 1.8 seconds, and reliable performance on standard CPU-only laptops. The principal trade-off observed is that SLM-generated responses tend toward concise phrasing and simpler synthesis of multi-step eligibility explanations compared to larger models.

3.2 Large Language Models (LLM)

Large Language Models, represented by Llama 3.2 (3B/8B variants) and Qwen 2.5, offer superior reasoning capability, including the ability to explain multi-step eligibility criteria and produce richer, more dynamic citizen-facing advice. However, this comes at the cost of higher hardware requirements — approximately 8–16 GB of RAM — and higher response latency, measured at approximately 3.6 seconds for Llama 3.2 in the project's benchmarking (see Chapter 6 for detailed results).

3.3 Retrieval-Only Mode (Fallback)

In addition to the SLM and LLM categories, CIVICASSIST implements a Retrieval-Only fallback mode that uses zero LLM inference, relying purely on the FAISS vector search and rule-based eligibility engine. This mode achieves ultra-fast response times of under 0.15 seconds, is 100% deterministic with zero hallucination risk, and requires minimal hardware (operable on 4 GB RAM CPU-only machines). The trade-off is that this mode displays exact scheme cards to the citizen without conversational, natural-language synthesis.

3.4 Comparative Summary

Table 3.1: Comparative Summary of SLM, LLM, and Retrieval-Only Modes

Attribute	Retrieval-Only	SLM (Phi-3 / Gemma 2B)	LLM (Llama 3.2 / Qwen 2.5)
Approx. RAM Requirement	4 GB	8 GB	16 GB
Approx. Response Latency	< 0.15 s	1.42 s – 1.85 s	~3.65 s
Hallucination Risk	None (deterministic)	Low	Low (with grounding)
Response Style	Structured scheme cards	Concise, simple synthesis	Rich, multi-step explanation
CPU-Only Friendly	Yes	Yes	Requires more capable hardware

3.5 Advantages and Disadvantages

Advantages of SLM

- Low latency, suitable for real-time citizen interaction.
- Lower hardware requirements, enabling deployment on standard laptops.
- Reduced energy and resource consumption relative to LLMs.

Disadvantages of SLM

- Less capable of nuanced, multi-step reasoning and explanation compared to LLMs.

Advantages of LLM

- Superior reasoning capability for explaining complex, multi-criteria eligibility rules.
- Richer, more natural conversational advice for citizens.

Disadvantages of LLM

- Higher RAM requirement (8–16 GB), which may not be available on low-end rural deployment hardware.
- Higher response latency (~3.65 seconds), which may affect user experience in high-throughput settings.

3.6 Justification for the Selected Approach

CIVICASSIST adopts a hybrid execution strategy rather than committing exclusively to either SLM or LLM inference. This design is justified by the benchmarking results (detailed in Chapter 6), which show that the Retrieval-Only mode achieves the highest factual groundedness score (0.95) at negligible latency, making it an ideal fallback and default for hardware-constrained deployments such as rural Common Service Centers. The SLM mode (Phi-3, groundedness 0.88) offers a practical balance between latency and response quality for interactive use, while the LLM mode (Llama 3.2, groundedness 0.92) is reserved for scenarios requiring richer explanation and where sufficient hardware resources are available. This tiered approach allows CIVICASSIST to adapt to varying deployment contexts — from resource-constrained kiosks to more capable desktop environments — without sacrificing the core requirements of privacy, groundedness, and zero recurring cost.

4. System Design

4.1 Overall Architecture

CIVICASSIST is architected as a local-first Retrieval-Augmented Generation (RAG) pipeline in which every processing stage — from citizen input capture to final response generation — executes entirely on the local device. The architecture integrates four principal subsystems: a citizen-facing input interface, a semantic retrieval subsystem built on FAISS and Sentence Transformers, a rule-based eligibility engine, and a local language-model inference engine served through Ollama. Figure 4.1 illustrates the end-to-end architecture and data flow of the system.

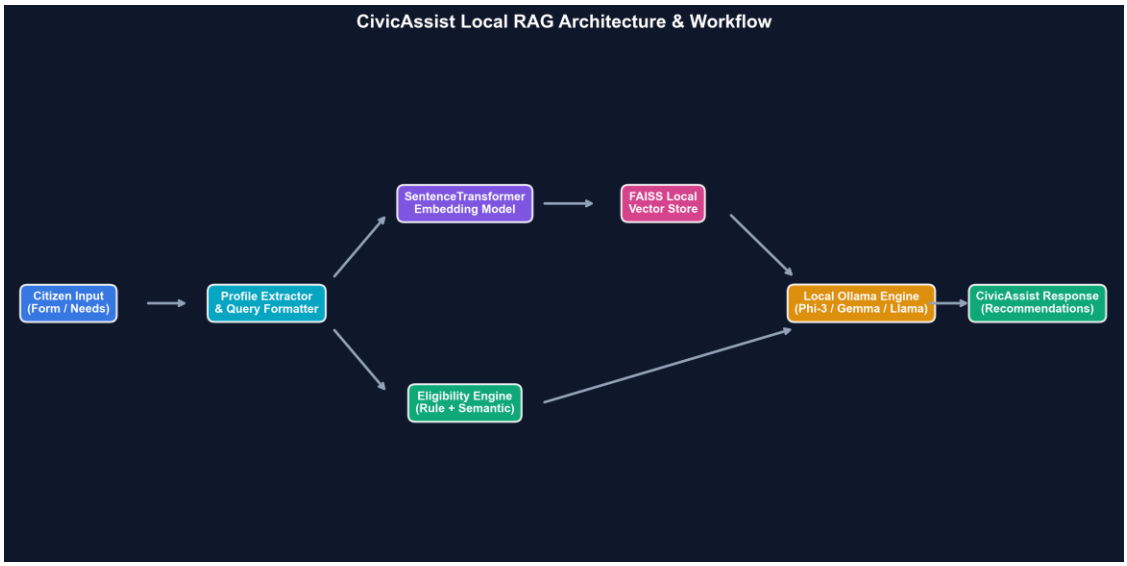


Figure 4.1: CivicAssist Local RAG Architecture and Processing Workflow

4.2 Workflow

As shown in Figure 4.1, the CivicAssist workflow proceeds through the following sequential stages:

- Citizen Input (Form / Needs): The citizen provides profile information and a description of their needs through the front-end interface.
- Profile Extractor & Query Formatter: The captured input is parsed and reformulated into (a) a semantic query for vector retrieval and (b) a structured profile object for eligibility scoring.

- Parallel Processing Branch 1 – Semantic Retrieval: The formatted query is passed to the SentenceTransformer Embedding Model, which converts it into a dense vector representation; this vector is then used to search the FAISS Local Vector Store for semantically relevant scheme candidates.
- Parallel Processing Branch 2 – Eligibility Engine: Simultaneously, the structured citizen profile is passed to the Eligibility Engine (Rule + Semantic), which computes a weighted match score for each candidate scheme.
- Local Ollama Engine: The outputs of both branches — semantically retrieved candidates and their eligibility scores — converge at the Local Ollama Engine, which hosts the Phi-3, Gemma, and Llama model family and generates the final natural-language response.
- CivicAssist Response (Recommendations): The system returns a citizen-facing response containing ranked, eligibility-verified scheme recommendations.

4.3 Knowledge Base

The CivicAssist knowledge base comprises a curated corpus of Central Government welfare schemes, each represented with structured metadata to support both semantic retrieval and rule-based eligibility scoring. Table 4.1 summarizes the core scheme corpus used in the prototype.

Table 4.1: Government Scheme Knowledge Base – Core Corpus

Scheme	Description
PM-KISAN	Direct income support for eligible farmer families.
Pradhan Mantri Awas Yojana (PMAY)	Housing assistance for EWS/LIG households.
Ayushman Bharat PM-JAY	Cashless health insurance up to ₹5 Lakh per family.
National Social Assistance Programme (NSAP)	Pension support for seniors, widows, and persons with disabilities.
Minority & Merit Scholarships	Education fee support for eligible students.
Women & Child Development Welfare	Nutrition and family welfare support.

Each scheme record in the knowledge base is stored with a metadata schema capturing the scheme name, description, eligibility criteria, benefits, category, applicable state, and official government source URL, ensuring that every citizen-facing recommendation can be traced back to an authoritative source.

4.4 FAISS (Facebook AI Similarity Search)

FAISS is used as the local vector store for semantic retrieval. Specifically, the system employs a FAISS IndexFlatL2 index, which performs exact nearest-neighbor search using L2 (Euclidean) distance over the embedded scheme vectors. This provides instant, exact similarity search suitable for the scale of the CivicAssist scheme corpus, and the vector store is generated automatically through an auto-indexing process on first application startup, requiring no manual configuration.

4.5 Sentence Transformers

Scheme text and citizen queries are converted into dense vector embeddings using the sentence-transformers/all-MiniLM-L6-v2 model, which produces 384-dimensional embeddings. This model was selected for its balance of embedding quality and computational efficiency, making it well suited for CPU-only, local deployment scenarios where larger embedding models would introduce unacceptable latency or memory overhead.

4.6 Ollama

Ollama serves as the local model-hosting and inference engine for CivicAssist, enabling Small and Large Language Models — Phi-3, Gemma, and Llama — to run entirely on-device without any dependency on external cloud inference APIs. By routing all natural-language generation through Ollama, the system guarantees that citizen queries and profile data are processed locally, in line with the project's zero-cloud, privacy-preserving design goal.

4.7 Streamlit

The citizen-facing interface is implemented using Streamlit, providing an interactive, form-based profile interface through which citizens enter their demographic and need-based information. Streamlit was selected for its ability to rapidly deliver a usable, browser-based interface suitable for both individual citizen use and kiosk-style deployment at rural digital service centers.

4.8 Rule-Based Eligibility Engine

The eligibility engine computes a quantitative match score for each candidate scheme using a weighted multi-factor formula:

$$\textit{Match Score} = (\Sigma \textit{ Field Scores} / \textit{Total Factors}) \times 100$$

Table 4.2 summarizes the weighted factors used in the eligibility scoring formula.

Table 4.2: Weighted Eligibility Scoring Factors

Factor	Weight / Points
Need Category Match (Healthcare, Housing, Agriculture, etc.)	100 points
Social Category Match (General, SC, ST, OBC, Minority)	100 points
Geographic Alignment (State / All India)	90 points
Income Qualification Check	+20 bonus points
Senior Citizen Age Check (≥ 60 years)	+20 bonus points

The engine operates in two stages. In Stage 1 (Semantic Retrieval), the FAISS index retrieves the Top-K candidate schemes matching the semantic context of the citizen's stated needs. In Stage 2 (Eligibility Filtering), each candidate is ranked according to its profile likelihood match on a 0–100% scale, with a threshold of ≥55% required for a scheme to be marked as "Recommended." This two-stage design ensures that only eligibility-verified matches are passed onward to the language model for explanation, which is the primary mechanism by which CivicAssist eliminates hallucinated scheme recommendations.

4.9 Retrieval-Augmented Generation (RAG)

The overall system implements the Retrieval-Augmented Generation paradigm by grounding every language-model response in retrieved, verified scheme data rather than relying on the model's parametric knowledge alone. By constraining the language model's generation to explain and present only the schemes that have already passed both the semantic retrieval and rule-based eligibility filtering stages, CivicAssist substantially reduces the risk of hallucinated content — a critical requirement given the civic and welfare-related nature of the information being presented to citizens.

5. Implementation

5.1 Software Requirements

The CivicAssist prototype is implemented as a Python-based application. Based on the architecture described in the project material, the core software components include the Streamlit framework for the front-end interface, the FAISS library for vector similarity search, the Sentence Transformers library for embedding generation, and Ollama for local SLM/LLM model serving.

5.2 Hardware Requirements

The project's deployment feasibility analysis (see Chapter 6, Section 6.5) specifies hardware requirements that scale with the selected execution mode. The Retrieval-Only mode requires approximately 4 GB of RAM, the SLM mode (Phi-3) requires approximately 8 GB of RAM, and the LLM mode (Llama 3.2) requires approximately 16 GB of RAM. The system was demonstrated as viable on standard consumer, CPU-only hardware, including Intel i5 and i7 class processors, without requiring a dedicated GPU.

5.3 Libraries Used

The following libraries and frameworks are explicitly identified in the project material as core to the CivicAssist implementation:

- FAISS – for local vector storage and similarity search (IndexFlatL2).
- sentence-transformers (all-MiniLM-L6-v2) – for generating 384-dimensional text embeddings.
- Streamlit – for the citizen-facing interactive web interface.
- Ollama – for local hosting and inference of the Phi-3, Gemma, and Llama model family.

5.4 Development Process

Based on the system architecture and workflow described in Chapter 4, the development process can be understood to have proceeded through the construction of the scheme knowledge base and metadata schema, generation of vector embeddings and auto-indexing into the FAISS store, implementation of the rule-based eligibility scoring engine, integration of the Ollama-hosted language models for response generation, and development of the Streamlit citizen interface.

5.5 Prompting Strategy

The prompting strategy adopted by CivicAssist is fundamentally retrieval-grounded: rather than allowing the language model to freely generate scheme recommendations, the system constrains generation to explain only the scheme candidates that have already been verified through the two-stage semantic-retrieval-and-eligibility-filtering pipeline described in Section 4.8. This grounding strategy is the primary mechanism by which the project reports the elimination of hallucinated scheme recommendations.

5.6 Model Selection

CivicAssist evaluates and supports multiple local models across the SLM and LLM categories: Phi-3 Mini (3.8B) and Gemma 2B as Small Language Models, and Llama 3.2 (3B/8B) and Qwen 2.5 as Large Language Models, in addition to a Retrieval-Only, zero-LLM fallback mode. As discussed in Chapter 3 and evaluated empirically in Chapter 6, model selection is guided by the trade-off between response latency, factual groundedness, and available hardware resources, with the project's benchmarking results supporting a hybrid, context-dependent model selection strategy rather than a single fixed model.

5.7 Transformer Architecture

The models employed in CivicAssist span two distinct transformer architecture categories, each suited to a different role within the pipeline:

Encoder-Only Architecture

The embedding model, sentence-transformers/all-MiniLM-L6-v2, is built on an encoder-only transformer architecture (a distilled variant of the BERT family). Encoder-only models are optimized for producing fixed-length, context-aware vector representations of input text rather than generating new text, which makes them well suited to the embedding-and-retrieval role they perform within the FAISS-based semantic search subsystem.

Decoder-Only Architecture

The response-generation models — Phi-3, Gemma, Llama 3.2, and Qwen 2.5 — are all decoder-only transformer models, the dominant architecture for autoregressive natural-language generation. Decoder-only models generate output tokens sequentially, conditioned on the retrieved

scheme context and citizen profile information, which is appropriate for CivicAssist's requirement to produce fluent, conversational explanations of eligibility results.

Justification

This architectural pairing — an encoder-only model for semantic embedding and retrieval, combined with decoder-only models for grounded response generation — reflects standard practice in Retrieval-Augmented Generation systems, where the retrieval and generation responsibilities are deliberately separated and handled by architectures optimized for each respective task.

6. Experimental Results

6.1 Test Cases

The CivicAssist prototype was demonstrated using three representative citizen personas, each designed to exercise a different combination of eligibility factors within the system. Table 6.3 summarizes the test cases and their corresponding outputs.

Test Case 1: Small Farmer

Profile: Age 42, Monthly Income ₹15,000, Occupation – Farmer, State – Karnataka, Stated Need – Agriculture & Income support.

Output Match: PM-KISAN (95% Match); PM-JAY Health Insurance (78% Match).

Test Case 2: Senior Citizen

Profile: Age 67, Monthly Income ₹8,000, Occupation – Retired, Stated Need – Pension.

Output Match: National Social Assistance Programme, NSAP (92% Match); Ayushman Bharat (85% Match).

Test Case 3: Minority Student

Profile: Age 19, Category – Minority, Occupation – Student, Stated Need – Education.

Output Match: Scholarship for Minorities (94% Match), with an official government portal citation provided alongside the recommendation.

Table 6.3: Test Case Summary – Prototype Demonstration

Test Case	Profile Summary	Top Match (Score)
TC1 – Small Farmer	Age 42, ₹15,000/mo, Farmer, Karnataka	PM-KISAN (95%)
TC2 – Senior Citizen	Age 67, ₹8,000/mo, Retired	NSAP (92%)
TC3 – Minority Student	Age 19, Minority, Student	Scholarship for Minorities (94%)

6.2 Performance Evaluation

The project benchmarked the four execution configurations — Retrieval-Only, Gemma 2B (SLM), Phi-3 3.8B (SLM), and Llama 3.2 3B (LLM) — across two dimensions: average response latency and factual groundedness / accuracy score. Figure 6.1 presents this comparison.

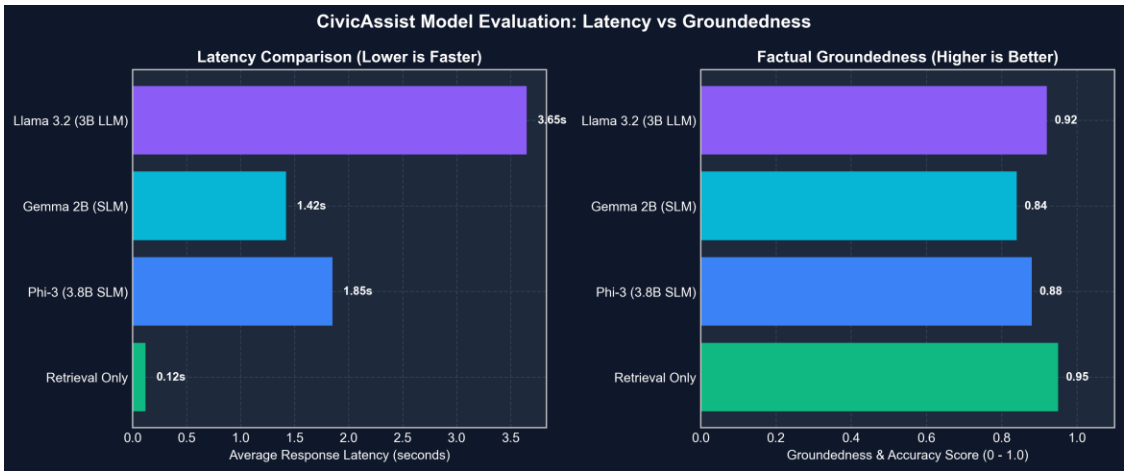


Figure 6.1: Model Evaluation – Latency vs. Factual Groundedness Comparison

6.3 Latency Comparison

Table 6.1: Response Latency and Factual Groundedness by Model

Model / Mode	Avg. Response Latency	Groundedness & Accuracy Score (0–1.0)
Retrieval Only	0.12 s	0.95
Gemma 2B (SLM)	1.42 s	0.84
Phi-3 (3.8B SLM)	1.85 s	0.88
Llama 3.2 (3B LLM)	3.65 s	0.92

The Retrieval-Only mode is the fastest configuration by a substantial margin, responding in approximately 0.12 seconds, while the Llama 3.2 LLM mode is the slowest at approximately 3.65 seconds — roughly 30 times slower than Retrieval-Only. Among the SLM options, Gemma 2B is marginally faster than Phi-3 (1.42s vs. 1.85s).

6.4 Accuracy Discussion

Somewhat counter-intuitively, the Retrieval-Only mode achieves the highest factual groundedness score (0.95) among all four configurations, which is consistent with its deterministic, non-generative nature — it returns exact scheme data without any risk of language-model hallucination. Among the generative configurations, Llama 3.2 achieves the highest groundedness score (0.92), followed by Phi-3 (0.88) and Gemma 2B (0.84). This suggests that, within the LLM/SLM categories, larger and more capable models produce comparatively more grounded responses when operating on the same retrieval-constrained context, though all generative configurations score lower than the purely deterministic Retrieval-Only baseline.

6.5 Deployment Feasibility

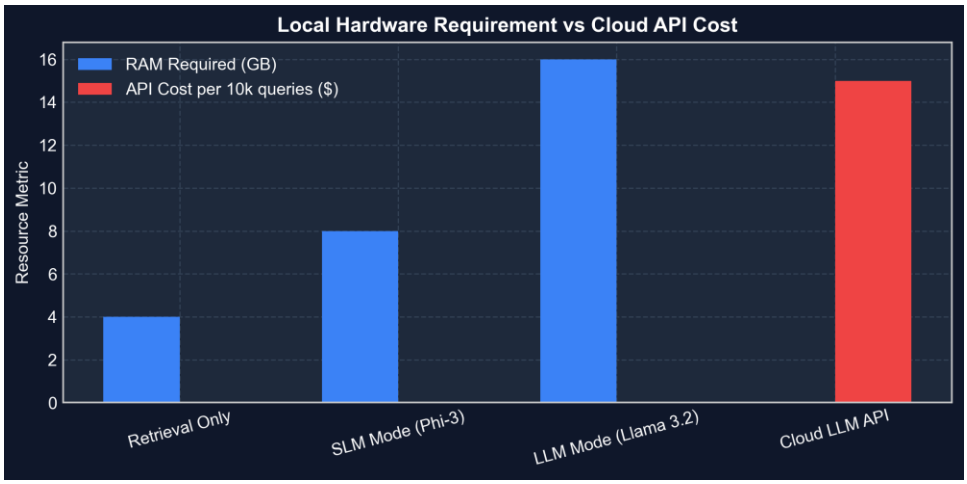


Figure 6.2: Local Hardware Requirement vs. Cloud API Cost Comparison

As shown in Figure 6.2, the hardware requirements scale progressively across the local execution modes: 4 GB of RAM for Retrieval-Only mode, 8 GB for SLM mode (Phi-3), and 16 GB for LLM mode (Llama 3.2). The project demonstrates that all three local configurations remain viable on standard consumer hardware (Intel i5/i7-class processors, CPU-only, 8–16 GB RAM), supporting deployment scenarios such as single-desktop installers or local servers for rural digital service centers, including Kiosks and Common Service Centers (CSCs).

6.6 Cost Analysis

Table 6.2: Hardware and Cost Comparison

Configuration	RAM Required (GB)	API Cost per 10,000 Queries
Retrieval Only	4	$0 (local)
SLM Mode (Phi-3)	8	$0 (local)
LLM Mode (Llama 3.2)	16	$0 (local)
Cloud LLM API	N/A (server-side)	$15+

As illustrated in Figure 6.2, commercial cloud LLM APIs incur a cost of $15 or more per 10,000 queries, whereas all three CivicAssist local execution modes incur zero recurring API expense, with the trade-off being a one-time local hardware investment (RAM capacity) rather than an ongoing per-query charge.

7. Discussion

7.1 Advantages

- Complete Data Privacy: Since all processing — embedding, retrieval, eligibility scoring, and language-model inference — occurs on the local device, no citizen data is ever transmitted to external servers.
- Zero Recurring Cost: The elimination of per-query cloud API charges (as demonstrated in Section 6.6) makes the platform economically sustainable for large-scale or high-frequency civic deployment.
- Low Resource Footprint: The system's ability to run in Retrieval-Only or SLM mode on 4–8 GB RAM CPU-only machines broadens its deployability to resource-constrained environments.
- Factual Grounding and Reduced Hallucination: The two-stage retrieval-and-eligibility-filtering design (Section 4.8, 4.9) ensures that only verified scheme matches are ever presented to citizens, directly addressing the hallucination risk identified in commercial cloud LLMs.
- Model Flexibility: The hybrid SLM/LLM/Retrieval-Only architecture allows deployers to select the configuration best suited to their available hardware and latency requirements.

7.2 Limitations

- Response Depth Trade-off: SLM and Retrieval-Only modes, while fast and resource-efficient, provide more concise or purely structured output compared to the richer, multi-step explanations available in LLM mode.
- Hardware Ceiling for LLM Mode: The 16 GB RAM requirement for LLM mode may exceed the capacity of low-end hardware commonly found in some rural deployment contexts.
- Fixed Knowledge Base: The current prototype operates on a static, manually curated scheme corpus (six schemes) rather than a live, continuously updated feed from official government sources; automated scheme scraping is identified as future work (Chapter 9) rather than a current capability.

- Single-Language Interface: The current implementation supports English-language interaction only; multi-lingual support is planned as a future enhancement rather than an existing feature.
- Limited Corpus Breadth: The demonstrated knowledge base covers six representative scheme categories; a production deployment would require substantially broader scheme coverage.

7.3 Practical Applications

- Rural Digital Service Centers and Common Service Centers (CSCs), where CivicAssist can be deployed as a local kiosk application without recurring connectivity or subscription costs.
- Government welfare offices, where staff could use the system to quickly cross-reference citizen eligibility across multiple schemes during in-person consultations.
- NGO and civil-society outreach programs operating in low-connectivity regions, where offline-capable, privacy-preserving scheme discovery could support field-level welfare access initiatives.

8. Conclusion

The CivicAssist project demonstrates that a fully local, privacy-preserving Retrieval-Augmented Generation pipeline — combining FAISS-based semantic search, a structured rule-based eligibility engine, and Ollama-hosted Small and Large Language Models — is a technically feasible and economically viable approach to government welfare scheme discovery, without dependency on any paid cloud API.

Three key findings emerge from this work. First, local RAG is fully feasible: the project's benchmarking results (Chapter 6) confirm that accurate government scheme matching can be achieved without any paid API dependency, and at zero recurring per-query cost compared to commercial cloud LLM alternatives that charge $15 or more per 10,000 queries. Second, the hybrid SLM/LLM model strategy is validated by the experimental results: Small Language Models such as Phi-3 provide an optimal balance of low latency (1.85 seconds) and good summary quality (0.88 groundedness score) for citizen guidance, while the Retrieval-Only mode remains the fastest and most deterministically grounded option for resource-constrained deployments. Third, the architecture achieves zero privacy risk, since citizen personal and demographic data remains strictly local to the client machine throughout the entire query-processing pipeline, from profile capture through final response generation.

Taken together, these findings support the central thesis of the project: that combining structured eligibility computation with locally hosted language models can deliver a citizen-facing welfare-scheme assistant that is simultaneously more private, more cost-effective, and more resistant to hallucination than existing government portal or commercial cloud LLM alternatives, while remaining deployable on modest, CPU-only consumer hardware.

9. Future Work

Building on the current prototype, the project identifies four directions for future enhancement:

- Live Scheme Scraping: Implementing automated scraping of official portals such as myscheme.gov.in, with active/expired status tracking, to keep the knowledge base current without manual curation.
- Multi-Lingual Interface: Extending the citizen-facing interface to support voice and text interaction in Hindi, Kannada, and other regional languages, improving accessibility for non-English-speaking citizens.
- Document Requirement Verification: Developing an automated checklist generator that identifies the specific documents a citizen must prepare for a given scheme application.
- Offline Mobile App Packaging: Converting the system into a mobile application capable of running on Android devices via embedded ONNX models, extending deployment beyond desktop and kiosk environments to individual citizen smartphones.

10. References

[1] J. Johnson, M. Douze, and H. Jégou, *FAISS: A Library for Efficient Similarity Search and Clustering of Dense Vectors*. Facebook AI Research. Available: https://github.com/facebookresearch/faiss

[2] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019. Available: https://www.sbert.net/

Embedding Model:

https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

[3] Ollama Team, *Ollama Documentation*. Available: https://ollama.com/

Documentation:

https://github.com/ollama/ollama

[4] Microsoft, *Phi-3 Technical Report*. Available:

https://huggingface.co/microsoft/Phi-3-mini-4k-instruct

Technical Report:

https://arxiv.org/abs/2404.14219

[5] Google DeepMind, *Gemma Open Models Documentation*. Available:

https://ai.google.dev/gemma

Model Card:

https://huggingface.co/google/gemma-2b

[6] Meta AI, *Llama 3.2 Model Collection*. Available:

https://www.llama.com/

Model Cards: https://huggingface.co/meta-llama

[7] Alibaba Cloud, *Qwen2.5 Large Language Models*. Available:

https://qwenlm.github.io/

Model Collection:

https://huggingface.co/Qwen

[8] Lewis, P., et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Paper:

https://arxiv.org/abs/2005.11401

[9] Government of India, *MyScheme – Government Schemes Portal*. Available: https://www.myscheme.gov.in/

[10] Government of India, *PM-KISAN Samman Nidhi*. Available: https://pmkisan.gov.in/

[11] Government of India, *Pradhan Mantri Awas Yojana (PMAY)*. Available: https://pmaymis.gov.in/

[12] National Health Authority, Government of India, *Ayushman Bharat Pradhan Mantri Jan Arogya Yojana (PM-JAY)*. Available: https://pmjay.gov.in/

[13] Ministry of Rural Development, Government of India, *National Social Assistance Programme (NSAP)*. Available: https://nsap.nic.in/

[14] Hugging Face, *Transformers Documentation*. Available: https://huggingface.co/docs/transformers

[15] Streamlit Inc., *Streamlit Documentation*. Available: https://docs.streamlit.io/

11. Appendices

A. Appendix A: Team Division of Responsibilities

Table 11.1: Team Division of Responsibilities

Team Member	Responsibilities
Anish M (2303917720521015)	Lead Engineer & Research – Architecture & RAG Pipeline Design, FAISS Vector Store Integration, System Validation & Benchmarking, Documentation & Report Writing
Moulesh G S (2303917720521087)	Lead Engineer & Research – Architecture & RAG Pipeline Design, FAISS Vector Store Integration, System Validation & Benchmarking, Documentation & Report Writing
Prasanna T (2303917720521107)	Model & Inference Optimization – Ollama API Integration & Tag Resolving, SLM vs LLM Performance Testing, Memory & Fallback Handling, Latency Profiling
Yogesh T (2303917720521177)	Frontend & UX Design – Streamlit UI Layout & Forms, Citizen Profile Schema, Scheme Card Components & Source Links, Error Notifications